DEVOPS WINTER ARC

90 Days Challenge • Kubernetes Week 3

DAY 20

Health, Resources & Autoscaling

MISSION: Make an app self-checking, resource-aware, and able to scale from 1 to 5 Pods under load.

Focus	Startup/readiness/liveness probes • CPU/memory requests & limits • QoS • metrics-server • HPA
Time	3–4 hours | 30% concepts + 70% hands-on
Deliverable	A Deployment with probes and resource settings that scales from 1 to 5 replicas under load.

1. Scope rule — no repetition

Day 15: Kubernetes basics and initial Nginx workload. Day 16: rollouts and rollback. Day 17: Service networking and DNS. Day 18: ConfigMaps, Secrets, and namespaces. Day 19: PV/PVC, StatefulSets, and persistent database storage. **Do not repeat these tasks.** Day 20 focuses on health probes, resource requests/limits, QoS, metrics-server, and HorizontalPodAutoscaler (HPA).

2. Concepts to know

Readiness: should this Pod receive traffic? Failure removes it from Service endpoints, but does not itself restart the container. **Liveness:** is the container stuck? Repeated failure causes a restart. **Startup:** gives slow apps time to initialize before other probes take over.
Requests guide scheduling and CPU utilization calculations. **Limits** cap resources; CPU can be throttled and memory overuse can cause OOMKilled. **QoS:** Guaranteed, Burstable, BestEffort. **HPA:** adjusts replicas using metrics and a target.

3. Task A — check metrics

```
kubectl get nodes
kubectl top nodes
kubectl top pods
```

If metrics are unavailable, install metrics-server for kind using the current official manifest, then add the kind-specific kubelet TLS setting. Verify metrics before creating the HPA.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl patch deployment metrics-server -n kube-system --type=json
-p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
kubectl rollout status deployment/metrics-server -n kube-system
kubectl top nodes
```

4. Task B — deploy a resource-aware workload

Create **day20-health/** and save the following as **deployment.yaml**. CPU requests are essential for CPU utilization-based HPA.

```
kubectl create namespace day20-health
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: php-apache
namespace: day20-health
spec:
replicas: 1
selector:
matchLabels:
app: php-apache
template:
metadata:
labels:
app: php-apache
spec:
containers:
- name: php-apache
image: registry.k8s.io/hpa-example
ports:
- containerPort: 80
resources:
requests:
cpu: 200m
memory: 64Mi
limits:
cpu: 500m
memory: 128Mi
startupProbe:
httpGet: {path: /, port: 80}
failureThreshold: 30
periodSeconds: 2
readinessProbe:
httpGet: {path: /, port: 80}
```

```
initialDelaySeconds: 5
periodSeconds: 5
livenessProbe:
httpGet: {path: /, port: 80}
initialDelaySeconds: 15
periodSeconds: 10
```

```
kubectl apply -f deployment.yaml
kubectl rollout status deploy/php-apache -n day20-health
kubectl describe pod -n day20-health -l app=php-apache
```

5. Task C — inspect resource usage and QoS

```
kubectl get pods -n day20-health
kubectl get pod -n day20-health -l app=php-apache -o jsonpath='{.items[0].status.qosClass}'
echo
kubectl describe pod -n day20-health -l app=php-apache
kubectl top pods -n day20-health
```

Expected QoS: **Burstable**, because requests are set but requests and limits are not equal for all CPU/memory resources. Record observed CPU and memory.

6. Task D — configure HPA

```
kubectl autoscale deployment php-apache -n day20-health \
--cpu-percent=50 --min=1 --max=5
kubectl get hpa -n day20-health
kubectl describe hpa php-apache -n day20-health
```

The target is 50% CPU utilization relative to the CPU request. HPA relies on observed metrics and reconciliation, so scaling is not instant.

7. Task E — generate load and watch scaling

Run this load generator in a terminal and leave it running temporarily:

```
kubectl run load-generator -n day20-health \
--image=busybox:1.36 --restart=Never -it --rm -- \
/bin/sh -c "while true; do wget -q -O- http://php-apache.day20-health.svc.cluster.local; done"
```

In a second terminal, observe HPA and replica changes:

```
kubectl get hpa -n day20-health -w
kubectl get deployments,pods -n day20-health -w
kubectl top pods -n day20-health
```

Success: under sustained CPU load, replicas rise above 1 but never exceed 5. After stopping the load, observe whether replicas eventually scale down. Save screenshots of HPA metrics and replica changes.

8. Task F — test a probe failure safely

Temporarily change the readiness probe path in the YAML to a nonexistent path, apply it, and observe the Pod become NotReady. Restore the path to */* afterward. Avoid leaving a broken liveness probe running, as it can cause repeated restarts.

```
kubectl apply -f deployment.yaml
kubectl describe pod -n day20-health -l app=php-apache
kubectl get events -n day20-health --sort-by=.lastTimestamp
```

Explain in your README: readiness failure affects traffic eligibility; liveness failure can restart the container; startup probes protect slow-starting apps.

9. Troubleshooting

Symptom	Check
kubectl top fails	Check metrics-server Pods, logs, and node metrics.
HPA shows unknown	Verify metrics-server and CPU requests; wait for metrics.
Replicas do not increase	Check HPA current/target, load, events, and maxReplicas.
Pod keeps restarting	Inspect Events, Last State, probe path/port, and logs.
OOMKilled	Inspect memory usage and limits.

10. Interview questions — answer out loud

Try each answer in 20–30 seconds before reading the suggested answer. Explain the reason and production impact.

Q1. Readiness vs liveness?
Suggested answer: Readiness decides if a Pod receives traffic; liveness detects a stuck container and can trigger a restart.

Q2. Why use a startup probe?
Suggested answer: It gives a slow-starting app time to initialize before liveness/readiness checks act.

Q3. What happens when readiness fails?
Suggested answer: The Pod becomes not ready and is removed from Service endpoints; readiness failure alone does not restart it.

Q4. What happens when liveness repeatedly fails?
Suggested answer: The kubelet restarts the container according to the probe and restart policy.

Q5. CPU request vs limit?
Suggested answer: Request is used for scheduling and utilization calculations; limit caps CPU, with excess use throttled.

Q6. What can memory limit exceedance cause?
Suggested answer: The container may be terminated with OOMKilled; memory limits should reflect measured needs.

Q7. Why does HPA need CPU requests?
Suggested answer: CPU utilization is measured relative to requested CPU; without requests, utilization may be undefined.

Q8. HPA vs manual scaling?
Suggested answer: Manual scaling sets replicas explicitly; HPA adjusts them continuously from metrics within configured bounds.

Q9. What does metrics-server do?
Suggested answer: It collects CPU/memory metrics from kubelets and exposes them to tools such as kubectl top and HPA.

Q10. Explain QoS classes.
Suggested answer: Guaranteed has equal CPU/memory requests and limits for every container; Burstable has some requests/limits but doesn't meet Guaranteed rules; BestEffort has none.

Q11. At 100% CPU target 50%, with 2 replicas, what is the rough desired count?
Suggested answer: 2 × 100 / 50 = 4, before bounds and other controller behavior.

Q12. Why might HPA not scale an app that feels slow?
Suggested answer: CPU may not be high, metrics may be missing, requests absent, max replicas reached, or the bottleneck may be elsewhere.

Q13. Should liveness fail whenever a database dependency is unreachable?
Suggested answer: Usually not by default; transient dependency failures can cause unnecessary restarts. Design checks around the app's actual failure modes.

Q14. Does autoscaling guarantee better performance?
Suggested answer: No. Node capacity, memory, downstream services, or non-CPU bottlenecks can still constrain performance.

11. Mini challenge & GitHub submission

■ Metrics work. ■ Deployment has requests, limits, and all three probes. ■ QoS identified. ■ HPA target/min/max documented. ■ Load makes replicas rise above 1 and never above 5. ■ Readiness failure observed and restored. ■ README includes troubleshooting and interview answers.

Create **day20-health-resources-autoscaling/** with **deployment.yaml**, **README.md**, and screenshots of metrics, QoS, HPA scaling, and probe troubleshooting. Do not repeat prior-day tasks.

12. Cleanup

```
kubectl delete namespace day20-health
```

If you installed metrics-server for this lab and want it for later Winter Arc work, keep it. Do not remove a shared cluster component blindly.

Final goal: Explain how Kubernetes checks app health, manages resource usage, classifies QoS, and scales workloads based on CPU metrics.